🔎 CEH v13 — Module 05: Vulnerability Analysis

> **Personal lab notes & command reference** | Certified Ethical Hacker (C|EH) v13
> Documented for learning and revision purposes. No exam answers or proprietary content reproduced.

📌 What is Vulnerability Analysis?

After scanning and enumeration, you have a map of the target — open ports, running services, OS versions, and usernames. Vulnerability analysis is the next step: **matching what you found against known weaknesses**.

A vulnerability is a **flaw in design or implementation** of a system that can be exploited to compromise its security. Two root causes account for nearly all vulnerabilities: software/hardware misconfiguration and poor programming practices.

Vulnerability assessment answers:

- Does this service have known CVEs?
- Are there missing patches?
- Are there weak authentication or encryption configurations?
- Are there default credentials or unnecessary services exposed?

> Vulnerability scanning ≠ exploitation. It identifies *what could be exploited*. The next phase (System Hacking) is where exploitation happens.

Two approaches to scanning:

- **Active Scanning** — sends probes directly to the target; generates traffic and may be detected
- **Passive Scanning** — observes traffic and behavior without direct interaction; stealthier but less complete

---

🗂️ Labs Index

#	Lab	Tool	Purpose
1	Vulnerability Research — CWE	cwe.mitre.org	Research known weaknesses by category/service
2	Vulnerability Assessment — OpenVAS	OpenVAS / Greenbone	Automated network vulnerability scanner

#	Lab	Tool	Purpose
3	Vulnerability Analysis — AI	ShellGPT + Nikto/Skipfish	AI-driven vuln scanning via natural language

---

Lab 1: Vulnerability Research using CWE

🧠 Concept

CWE (Common Weakness Enumeration) is a community-developed list of software and hardware weakness types. It's maintained by MITRE and serves as the backbone for categorizing the *root causes* of vulnerabilities — not specific CVEs, but the underlying weakness patterns that lead to them.

Think of it as:

- **CVE** = a specific vulnerability in a specific product (e.g., EternalBlue in Windows SMB)
- **CWE** = the weakness category that CVE exploits (e.g., CWE-119: Buffer errors)

CWE is used to:

- Research what classes of weaknesses affect a protocol/service you've enumerated
- Map CVEs back to their root causes
- Understand what defensive controls address a category of weakness

Key CWE Views:

- **CWE Top 25** — the 25 most dangerous software weaknesses annually (most exploited in the wild)
- **OWASP Top 10** — web-specific weakness mapping
- **Research Concepts / Development Concepts** — different angles on the same weaknesses

---

⚙️ Research Workflow

Step 1: Search for weaknesses by service

```
https://cwe.mitre.org → Search → "SMB"
```

> **Why:** After enumerating SMB in Module 4, you search CWE for SMB-related weaknesses to understand what classes of vulnerabilities this service is historically prone to.

Example results for SMB:

CWE ID	Name	Relevance
CWE-284	Improper Access Control	SMB shares with wrong permissions
CWE-319	Cleartext Transmission of Sensitive Info	SMBv1/v2 without encryption
CWE-552	Files Accessible to External Parties	World-readable NFS/SMB shares
CWE-200	Exposure of Sensitive Information	Information disclosure via SMB

Step 2: View the CWE Top 25 (2023)

```
https://cwe.mitre.org → CWE List → External Mappings → CWE Top 25 (2023)
```

2023 CWE Top 25 — Most Dangerous Software Weaknesses:

Rank	CWE ID	Weakness Name
1	CWE-787	Out-of-bounds Write
2	CWE-79	Cross-site Scripting (XSS)
3	CWE-89	SQL Injection
4	CWE-416	Use After Free
5	CWE-78	OS Command Injection
6	CWE-20	Improper Input Validation
7	CWE-125	Out-of-bounds Read
8	CWE-22	Path Traversal
9	CWE-352	Cross-Site Request Forgery (CSRF)
10	CWE-434	Unrestricted Upload of Dangerous File Type
11	CWE-862	Missing Authorization
12	CWE-476	NULL Pointer Dereference
13	CWE-287	Improper Authentication
14	CWE-190	Integer Overflow or Wraparound
15	CWE-502	Deserialization of Untrusted Data
16	CWE-77	Command Injection
17	CWE-119	Memory Buffer Restriction Errors
18	CWE-798	Use of Hard-coded Credentials
19	CWE-918	Server-Side Request Forgery (SSRF)
20	CWE-306	Missing Authentication for Critical Function
21	CWE-362	Race Condition
22	CWE-269	Improper Privilege Management
23	CWE-94	Code Injection
24	CWE-863	Incorrect Authorization
25	CWE-276	Incorrect Default Permissions

🔑 Key Takeaways

- CWE is a **research tool**, not a scanner — it teaches you what *type* of weakness to look for, not whether a specific system has it
- The CWE Top 25 changes yearly — bookmark it; it reflects what's actively exploited that year
- Searching by service (SMB, FTP, HTTP) after enumeration narrows your research to relevant weakness classes
- CWE IDs appear in CVE entries — knowing CWE helps you understand *why* a CVE exists, not just *that* it does
- **Vulnerability mapping abstraction levels:** Pillar → Class → Base → Variant (from broad to specific)

Lab 2: Vulnerability Assessment using OpenVAS

🧠 Concept

OpenVAS (Open Vulnerability Assessment System) is an open-source vulnerability scanner maintained by Greenbone Networks. It's the most widely used free alternative to commercial scanners like Nessus and Qualys.

OpenVAS works by running **Network Vulnerability Tests (NVTs)** — over 50,000 automated tests that probe targets for known vulnerabilities, misconfigurations, and weak settings.

Key capabilities:

- Unauthenticated scanning (external attacker perspective)
- Authenticated scanning (with credentials — much more thorough)
- Large-scale network scanning
- CVSS-scored results with remediation guidance

Important insight from the lab: Running OpenVAS with the firewall ON vs. OFF on the target produced the **same results**. This demonstrates that vulnerability scanners probe at the application/service level — a host firewall doesn't hide services that are already listening and responding.

🖥️ Environment

Role	Machine	IP
Scanner (Attacker)	Parrot Security (Linux)	10.10.1.13
Target	Windows Server 2022	10.10.1.22

⚙️ Setup & Commands

Launch OpenVAS via Docker:

```bash
docker run -d -p 443:443 --name openvas mikesplain/openvas
```

> **One-liner:** Pull and run the OpenVAS container, mapping the web interface to local port 443 — packages the entire OpenVAS stack (scanner + web GUI + NVT database) into a single Docker container for easy deployment.

Part	Meaning
`docker run -d`	Run container in background (detached)
`-p 443:443`	Map host port 443 to container port 443
`--name openvas`	Name the container for easy reference
`mikesplain/openvas`	Pre-built OpenVAS Docker image

Access the web interface:

```
https://127.0.0.1
Login: admin / admin
```

📋 OpenVAS Scan Workflow

```
Scans → Tasks → Wand icon → Task Wizard
  → Enter target IP: 10.10.1.22
    → Start Scan
```

```
→ Wait for Status: Done (~20 mins)
  → Click Done to view Report: Results
```

Task Wizard automatically:

1. Creates a new Target object for the IP
2. Creates a new Task with default scan config
3. Starts the scan immediately
4. Reloads every 30 seconds to show progress

📊 Understanding OpenVAS Results

Report columns:

Column	Meaning
Vulnerability	Name of the detected weakness
Severity	CVSS score + label (Critical/High/Medium/Low/Log)
QoD	Quality of Detection % — how confident OpenVAS is in the finding
Host	Target IP where the vulnerability was found
Location	Port/protocol (e.g., `135/tcp`)

CVSS Severity scale:

Score Range	Label	Meaning
9.0–10.0	Critical	Exploit immediately available, remote, no auth
7.0–8.9	High	Significant impact, likely exploitable
4.0–6.9	Medium	Requires conditions or local access
0.1–3.9	Low	Minor impact or difficult to exploit
0.0	Log	Informational — no direct risk

Example findings from lab scan of 10.10.1.22:

Vulnerability	Severity	Port	What It Means
DCE/RPC and MSRPC Services Enumeration Reporting	Medium (5.0)	135/tcp	RPC services are enumerable — UUID and endpoint info is exposed
TCP timestamps	Low (2.6)	general/tcp	TCP timestamps enabled — allows OS uptime estimation

---

🔬 Vulnerability Detail: DCE/RPC and MSRPC Enumeration

What it is: The target has port 135 open (Windows RPC endpoint mapper). By connecting to this port, an attacker can enumerate all registered RPC services including their UUIDs, endpoints, and version numbers.

Why it matters:

- RPC enumeration reveals internal service architecture
- Specific UUIDs can be matched to known exploits (e.g., MS03-026 used a vulnerable RPC UUID)
- No authentication required to enumerate

Port 135 RPC services exposed in lab:

```
Port: 2103/tcp
  UUID: 1088a980-eae5-11d0-8d9b-00a02453c337 → Message Queuing (MSMQ)
  UUID: 1a9134dd-7b39-45ba-ad88-44d01ca47f28 → Message Queuing RemoteRead
```

🔑 Key Takeaways

- OpenVAS = free Nessus alternative — essential tool to know for the exam and real engagements
- **QoD %** matters: 80%+ = high confidence; below 50% = needs manual verification
- Firewall ON vs. OFF = **same scan results** — a firewall doesn't make services disappear; it has to actually block them
- Authenticated scans (with credentials) return far more findings than unauthenticated scans
- OpenVAS results feed directly into the next phase: selecting exploits for high/critical findings

- Always manually verify Medium+ findings — false positives are common with automated scanners

---

Lab 3: Vulnerability Analysis using AI (ShellGPT)

🧠 Concept

ShellGPT extends its scanning capability into vulnerability analysis. While Module 3 used it for network/port scanning, here it's applied to **web application vulnerability scanning** using tools like Nikto and Skipfish.

Tools used through ShellGPT:

- **Nikto** — web server scanner; checks for dangerous files, outdated software, misconfigurations, and HTTP header issues
- **Skipfish** — active web application security scanner; crawls the site and categorizes findings by issue type

---

🖥️ Environment

Role	Machine	Target
Attacker	Parrot Security (Linux)	www.certifiedhacker.com, testphp.vulnweb.com

---

⚙️ Commands & Explanations

Configure ShellGPT:

```bash
bash sgpt.sh
# Enter AI Activation Key when prompted
```

Nikto scan via ShellGPT:

```bash
sgpt --chat nikto --shell "Scan the URL https://www.certifiedhacker.com to identify
# Type E to Execute
```

> **One-liner:** Ask ShellGPT to generate and run a Nikto web vulnerability scan — translates to `nikto -h https://www.certifiedhacker.com` and returns HTTP header issues, server fingerprint, outdated software versions, and risky file paths.

What the generated command does:

```bash
nikto -h https://www.certifiedhacker.com
```

Nikto output fields:

```
+ Target IP:          162.241.216.11
+ Target Hostname:    www.certifiedhacker.com
+ Target Port:        443
+ SSL Info:           Subject: /CN=webdisk.certifiedhacker.com
                      Ciphers: TLS_AES_256_GCM_SHA384
                      Issuer: Let's Encrypt

+ Server: nginx/1.25.5
+ /: The anti-clickjacking X-Frame-Options header is not present
+ /: Uncommon header 'x-proxy-cache' found, with contents: HIT
+ /: Uncommon header 'x-server-cache' found, with contents: true
+ /: The site uses TLS but Strict-Transport-Security header is not defined
+ /: The X-Content-Type-Options header is not set
```

What each finding means:

Finding	Security Impact
Missing `X-Frame-Options`	Site can be embedded in iframes → clickjacking attacks possible
Missing `Strict-Transport-Security`	Browser may fallback to HTTP → downgrade/MITM attacks
Missing `X-Content-Type-Options`	Browser may interpret files as different MIME types → content sniffing attacks
Uncommon response headers	Reveals proxy/caching infrastructure details → information disclosure
SSL certificate CN mismatch	CN doesn't match hostname → potential misconfiguration

> ⏱️ Nikto scans are slow — can take hours on large sites. Use `Ctrl+Z` to stop.

Nmap web vulnerability scan via ShellGPT:

```bash
sgpt --chat vuln --shell "Perform vulnerability scan on target url http://www.movies
# Type E to Execute
```

> **One-liner:** ShellGPT generates an Nmap command using the `http-vuln*` NSE scripts against port 80 of the target URL — checks for known HTTP-based vulnerabilities without a full crawl.

Generated command:

```bash
nmap --script http-vuln* -p 80 www.moviescope.com
```

Skipfish scan via ShellGPT:

```bash
sgpt --chat vuln --shell "Perform a vulnerability scan on target url http://testphp.
# Type E to Execute
```

> **One-liner:** ShellGPT generates a skipfish command that crawls the target web application, discovers all endpoints, and categorizes findings by issue type — results saved as an HTML report in `/tmp/skipfish_scan_output/index.html`.

Generated command:

```bash
skipfish -o /tmp/skipfish_scan_output http://testphp.vulnweb.com
```

Skipfish report issue categories:

Category	Risk	Example
Conflicting MIME/charset info	High	Content-Type header mismatches
HTML form with no XSRF protection	Medium	Forms submittable from other sites
External content embedded	Low	Third-party JS/CSS loaded without SRI
Resource fetch failed	Info	404/500 errors revealing path structure
Numerical filename — consider enumerating	Info	Predictable file names (file1.php, file2.php)

📋 Tool Comparison: Nikto vs. Skipfish vs. OpenVAS

Aspect	Nikto	Skipfish	OpenVAS
Target type	Web servers	Web applications	Network hosts + services
Approach	Signature-based checks	Active crawl + fuzzing	NVT-based port/service testing
Speed	Slow (hours)	Medium (minutes–hours)	Slow (20+ min per host)
Output format	Terminal / text	HTML report	Web GUI + XML
Authenticated?	Optional	No	Yes (much better with creds)
Best for	HTTP header issues, software versions	Web app logic flaws, form issues	Network-wide vuln mapping

---

🔑 Key Takeaways

- Nikto is the go-to for **quick web server checks** — especially HTTP security headers
- Missing security headers (HSTS, X-Frame-Options, CSP, X-Content-Type-Options) are consistently found and are easy remediation wins for blue teams
- Skipfish is more thorough for **web application logic** — it crawls and fuzzes, not just checks headers
- ShellGPT's `--chat` context means you can follow up: `sgpt --chat nikto --shell "now re-scan with SSL checks enabled"` — it remembers the previous command

- AI-generated vuln commands need verification before execution — review the generated command for correctness before pressing E

🛠️ Tools & Resources Quick Reference

Tool / Resource	Type	URL / Command	Best For
CWE	Database	https://cwe.mitre.org	Research weakness categories
CVE / NVD	Database	https://nvd.nist.gov	Specific vulnerabilities + CVSS scores
OpenVAS	Scanner	`docker run mikesplain/openvas`	Network host vulnerability scanning
Nikto	Scanner	`nikto -h <url>`	Web server misconfiguration
Skipfish	Scanner	`skipfish -o <dir> <url>`	Web application crawl + fuzz
Nmap NSE `http-vuln*`	Scanner	`nmap --script http-vuln* -p 80 <target>`	HTTP-specific known vulnerabilities

🔐 Vulnerability Severity — CVSS v3 Scoring Explained

CVSS (Common Vulnerability Scoring System) v3 scores are based on:

Base Score factors:

Metric	Options	Weight
Attack Vector	Network / Adjacent / Local / Physical	High
Attack Complexity	Low / High	Medium
Privileges Required	None / Low / High	Medium
User Interaction	None / Required	Low
Scope	Unchanged / Changed	Medium
Confidentiality Impact	None / Low / High	High
Integrity Impact	None / Low / High	High
Availability Impact	None / Low / High	High

> A vulnerability scored `AV:N/AC:L/PR:N/UI:N` = **Network-accessible, low complexity, no privileges, no user interaction** = almost certainly a 9.0+ Critical. This is the worst-case scenario for defenders.

🔐 Defensive Takeaways (Blue Team Perspective)

Vulnerability Class	Detection	Mitigation
Missing HTTP security headers	Nikto scan / header inspection	Add HSTS, X-Frame-Options, CSP, X-Content-Type-Options
RPC/DCOM exposure (port 135)	OpenVAS / nmap	Firewall port 135 from external; disable unused RPC services
TCP timestamp disclosure	OpenVAS	`sysctl -w net.ipv4.tcp_timestamps=0` (Linux)
Cleartext protocol exposure	Nikto / Wireshark	Enforce TLS; disable HTTP, Telnet, FTP
CWE-798 (hardcoded credentials)	Code review / static analysis	Secrets management (Vault, AWS Secrets Manager)
CWE-79 XSS	DAST scanners / Skipfish	Input sanitization, output encoding, CSP headers
CWE-89 SQLi	Skipfish / sqlmap	Parameterized queries, ORM, WAF

Vulnerability Class	Detection	Mitigation
Missing patches (OpenVAS findings)	OpenVAS authenticated scan	Patch management cadence; auto-updates for critical patches

---

📎 Notes

- **Lab environment:** CEH v13 iLabs (EC-Council official)
- **Target IPs:** 10.10.1.22 (Windows Server 2022)
- **External targets:** www.certifiedhacker.com, testphp.vulnweb.com (authorized lab targets)
- All techniques performed in a **controlled, authorized lab environment**

Module 05 complete. See Module 04 — Enumeration | Module 03 — Scanning